

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ защищённости программных систем.
Часть 2. Оценка защищённости сторонних программных систем»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: разработать методику комплексной оценки защищённости сторонних программных систем и применить её на практике к реальному приложению, действуя со стороны внешнего наблюдателя («чёрный ящик»), а не со стороны разработчика системы.

Задание:

- а) Разработать методику оценки защищённости систем.
- б) Выполнить анализ предоставленного приложения с позиций: политики безопасности операционной системы; политики безопасности компонентов приложения и их взаимодействия между собой; сетевой политики безопасности; безопасности конфиденциальности данных.
- в) Предложить комплекс мер по усовершенствованию приложения.
- г) Предложить рекомендации по безопасному использованию приложения.
- д) Оформить отчёт о проделанной работе.

Постановка задачи. В качестве «предоставленного приложения» (инструментальное средство №2 задания) выступает вариант, выполненный в рамках лабораторной работы №2.1, – веб-приложение **SecureApp (Lab2.1-SecureApp)**. Разница с лабораторной работой №2.1 принципиальна: там анализ велся от лица разработчика, знающего исходный код и намеренно закладывающего в него конкретные защитные механизмы; здесь тот же набор направлений анализа (ОС, компоненты и их взаимодействие, сеть, конфиденциальность) выполняется от лица внешнего аудитора – с помощью чёрного ящика и целенаправленных проверок, – а исходный код привлекается только для подтверждения или опровержения гипотез, выдвинутых по результатам чёрного ящика, а не как основной источник информации.

Краткое описание предоставленной системы

С точки зрения внешнего наблюдателя предоставленная система – это одна конечная точка HTTP (<http://localhost:5080>), отвечающая набором маршрутов вида `/api/auth/*`, `/api/confidential/*` и `/api/public/*`, и статический веб-клиент, обращающийся к этой точке. О внутреннем устройстве системы на этом этапе ничего не известно – ни стек технологий, ни СУБД, ни модель данных; всё перечисленное предстоит либо подтвердить, либо опровергнуть в ходе анализа. Косвенные признаки (формат ошибок валидации, поведение маршрутизации, заголовок **Server**) в дальнейшем позволяют установить, что это REST API на ASP.NET Core (сервер Kestrel) с хранением данных в SQLite и клиентом на React – то есть тот же вариант лабораторной работы №2.1 (**Lab2.1-SecureApp**), что и указано в задании.

Разработанная методика оценки защищённости

Методика построена на семи направлениях анализа, перечисленных в теоретических основах задания, и для каждого направления определяет конкретную технику проверки в духе OWASP Testing Guide, реализованную как самостоятельный скрипт (`audit.sh`) вместо готового инструмента (`ZAP/nikto/sqlmap/gobuster` в среде выполнения недоступны) – каждый скрипт выполняет ту же категорию проверки, что и соответствующий инструмент, но по фиксированному, воспроизводимому сценарию.

а) **Анализ физических устройств хранения.** Определение, где и в каком виде физически хранятся данные (файл СУБД, права доступа к нему на уровне файловой системы), без обращения к исходному коду.

б) **Анализ ОС и программного окружения.** Определение стека (по заголовку `Server`, формату ошибок, поведению маршрутов) и оценка того, какие секреты (ключи, пароли) требуются окружению и как они предположительно поставляются.

в) **Разведка и поиск ресурсов (WSTG-INFO).** Перебор типовых путей (`robots.txt`, `.env`, `.git/*`, `swagger`, файлы резервных копий), проверка нестандартных HTTP-методов (`TRACE`), поиск случайно опубликованной документации API (`OpenAPI/Swagger`).

г) **Анализ сетевой политики.** Проверка заголовков, относящихся к транспортной безопасности (`Strict-Transport-Security`, `Content-Security-Policy`, `CORS`), и фактического наличия принудительного HTTPS.

д) **Инъекционное тестирование (WSTG-INPV).** Отправка SQL-инъекционных полезных нагрузок (в том числе разрушающих – `DROP TABLE`) в поля входа, использующие пользовательский ввод, с последующей проверкой контрольной записи-маркера (`sanapu`), созданной до атаки: если маркер исчез или таблица недоступна – инъекция удалась.

е) **Тестирование межсайтового скриптинга (XSS).** Сохранение полезной нагрузки `<script>/<img onerror>` через API и проверка, экранирует ли сам API вывод, независимо от того, как эти данные далее отображает конкретный клиент.

ж) **Анализ аутентификации и сессий.** Проверка устойчивости к типовым атакам на JWT (алгоритм `none`, подпись отделена от токена), проверка одинаковости политики ограничения частоты запросов на всех точках входа, где возможен подбор (не только `login`, но и `register`), проверка атрибутов `cookie` сессии.

з) **Анализ отказоустойчивости ввода.** Проверка, что входные данные ограничены по размеру (защита от простейшего DoS через большие полезные нагрузки).

и) **Анализ конфиденциальности данных.** Прямое чтение необра-

ботанного файла хранилища в поисках контрольного значения, записанного через API как конфиденциальное – аналогично методике, применённой в лабораторной работе №2.1, но здесь выполняется как независимая, не полагающаяся на код проверка.

Пункты «анализ исходного кода» и «анализ организационных мер» из теоретических основ задания выполняются отдельно от `audit.sh`: исходный код (доступный только потому, что предоставленная система – это наша же лабораторная работа №2.1) привлекается исключительно для объяснения причины уже обнаруженного чёрным ящиком отклонения, а организационные меры (журналирование, регламент эксплуатации, хранение ключей) оцениваются по наблюдаемому поведению системы (например, по содержимому лог-файла процесса) там, где это возможно без доступа к коду.

Анализ защищённости предоставленной системы

1. Политика безопасности операционной системы

Чёрный ящик не может напрямую проверить права на файл базы данных или способ хранения переменных окружения – эти два свойства подтверждены по логам процесса и по документации приложения (`README.md` в составе поставки), а не инструментально. Обнаружено, что процесс сервера пишет в свой лог только служебные сообщения фреймворка: ни одна попытка входа, блокировки или регистрации не порождает ни одной прикладной записи в журнале (см. листинг 1) – то есть у системы полностью отсутствует журнал аудита событий безопасности, что затрудняет обнаружение атаки постфактум средствами ОС (например, анализом логов) даже при наличии самого механизма блокировки.

2. Политика безопасности компонентов приложения и их взаимодействия

Из девяти проверок в фазах 3–6 методики шесть завершились без замечаний и три выявили проблему. SQL-инъекционные нагрузки, включая деструктивную `DROP TABLE` и `UNION SELECT`, не нарушили работу системы: контрольная запись осталась читаемой после всех попыток (листинг 1, фаза 3) – признак корректно параметризованных запросов. Поддельный JWT с алгоритмом `none` и токен с обрезанной подписью были отклонены (фаза 5) – классическая уязвимость путаницы алгоритмов подписи отсутствует. Однако обнаружено, что конечная точка документации API (`/openapi/v1.json`) открыта без аутентификации и в открытом виде перечисляет все маршруты API, включая внутренние `/api/confidential/{id}` (листинг 1, фаза 1) – разработчик обычно ожидает, что этот маршрут доступен только в среде разработки, но чёрный ящик не видит значения переменной окружения, определяющей это условие, и обнаруживает лишь факт: маршрут отвечает. Отдельно обнаружено, что API не выполняет никакого серверного экранирования сохраняемого содержимого (фаза 4): полезная нагрузка `<script>alert(1)</script>` сохраняется и возвращается веб-сервисом без изменений. Разбор исходного кода клиента (предпринятый только для объяснения этой находки, а не как основной метод) показал, что поставляемый React-клиент нигде не использует `innerHTML` или `dangerouslySetInnerHTML` и поэтому не исполняет такую нагрузку в браузере – то есть по состоянию на сегодня уязвимость не эксплуатируема через штатный интерфейс, но остаётся скрытым риском для любого другого клиента (мобильного приложения, экспорта данных, административной панели), который в будущем станет отображать те же данные как HTML.

3. Сетевая политика безопасности

Заголовки `X-Content-Type-Options`, `X-Frame-Options` и `Referrer-Policy` присутствуют во всех ответах, тогда как `Strict-Transport-Security` и `Content-Security-Policy` отсутствуют полностью (листинг 1, фаза 2) – оба являются стандартными для современного веб-приложения рубежами защиты и обычно проверяются любым автоматическим сканером за секунды. Кроме того, в служебном логе процесса при каждом запросе фиксируется сообщение `Failed to determine the https port for redirect`, то есть перенаправление на HTTPS формально включено в коде (`UseHttpsRedirection`), но не может сработать, поскольку в запущенной конфигурации не привязана ни одна HTTPS-конечная точка – на практике это означает, что в развёрнутом виде (не на `localhost`) весь трафик, включая пароль при входе и cookie сессии, идёт открытым текстом, если перед приложением не поставлен отдельный узел терминирования TLS. Заголовок `Server: Kestrel` демаскирует используемый стек – малозначительная, но ненулевая помощь атакующему на этапе разведки. Политика CORS при этом подтверждена корректной повторно: посторонний источник не получает заголовка `Access-Control-Allow-Origin` вовсе, а метод `TRACE` сервером Kestrel не поддерживается (фаза 1) – эти два рубежа защиты выдержали проверку.

4. Безопасность конфиденциальности данных

Контрольное значение, записанное как конфиденциальное поле через публичный API, не найдено ни в каком виде в необработанном файле базы данных, включая WAL-журнал SQLite (листинг 1, фаза 7) – шифрование данных при хранении подтверждено чёрным ящиком независимо от исходного кода, что и было целью этой проверки. Отдельное наблюдение, уже требующее обращения к коду для объяснения: и ключ подписи токена сессии, и ключ шифрования конфиденциальных полей поступают из одной и той же категории конфигурационных значений процесса – то есть компрометация конфигурации разом ставит под удар и аутентификацию, и конфиденциальность данных, без разделения ключей по областям ответственности.

Полный протокол проверки, включающий все семь фаз методики, приведён ниже.

```

1 == Phase 1: Reconnaissance -- server banner and resource discovery ==
2 [FINDING] Server banner disclosed: 'Server: Kestrel' (aids an attacker in picking known CVEs for the stack)
3 [FINDING] OpenAPI schema publicly exposed at /openapi/v1.json (HTTP 200) -- discloses full API surface:
  ↳ "/api/auth/login" "/api/auth/logout" "/api/auth/me" "/api/auth/register" "/api/confidential"
  ↳ "/api/confidential/{id}" "/api/public" "/api/public/{id}"
4 [PASS] /robots.txt not exposed (HTTP 404)
5 [PASS] /.env not exposed (HTTP 404)
6 [PASS] /.git/config not exposed (HTTP 404)
7 [PASS] /.git/HEAD not exposed (HTTP 404)
8 [PASS] /swagger not exposed (HTTP 404)
9 [PASS] /swagger/index.html not exposed (HTTP 404)
10 [PASS] /.well-known/security.txt not exposed (HTTP 404)
11 [PASS] /backup.zip not exposed (HTTP 404)
12 [PASS] /web.config not exposed (HTTP 404)
13 [PASS] TRACE method not enabled (HTTP 404)
14 == Phase 2: Network / transport security headers ==
15 [PASS] X-Content-Type-Options header present
16 [PASS] X-Frame-Options header present
17 [PASS] Referrer-Policy header present
18 [FINDING] Strict-Transport-Security header missing
19 [FINDING] Content-Security-Policy header missing
20 == Phase 3: Injection testing (SQL injection, per OWASP Testing Guide WSTG-INPV) ==
21 [INFO] Created canary record #1 to detect a successful injection (e.g. a dropped/truncated table)
22 [PASS] Classic OR-based SQLi in login.username did not authenticate ({"message":"Invalid username or
  ↳ password."})
23 [PASS] Canary record #1 survives DROP TABLE / UNION SELECT payloads in the search parameter -- EF Core
  ↳ parameterization holds
24 == Phase 4: Stored XSS / output-encoding testing (OWASP WSTG-INPV-01/02) ==
25 [FINDING] API stores and echoes an XSS payload verbatim, with no server-side sanitization or output encoding
  ↳ (not currently exploitable through the shipped React frontend, which never uses
  ↳ innerHTML/dangerouslySetInnerHTML -- see source review in the report -- but is a latent risk for any other
  ↳ client)
26 == Phase 5: Authentication and session security ==
27 [PASS] Forged alg:none JWT rejected (HTTP 401) -- no JWT algorithm-confusion vulnerability
28 [PASS] Login endpoint rate-limited after 5 failures (HTTP 429)
29 [FINDING] Registration endpoint has NO rate limiting (8/8 unthrottled: 409 409 409 409 409 409 409 409 ) --
  ↳ allows unlimited username-enumeration probes, unlike the login endpoint
30 [PASS] Session cookie carries HttpOnly; Secure; SameSite=Strict
31 == Phase 6: Input validation / resource-exhaustion boundary ==
32 [PASS] Oversized content (10050 chars) rejected by validation (HTTP 400)
33 == Phase 7: Confidentiality -- raw storage inspection ==
34 [PASS] Confidential marker not found in plain text anywhere in the raw database file (record #1)
35 === Summary ===
36 Checks: 26 total, 6 findings

```

Листинг 1 – Реальный протокол аудита SecureApp (скрипт audit.sh): 26 проверок, 6 замечаний

Комплекс мер по усовершенствованию защищённости

а) **Скрыть документацию API в эксплуатации.** Публикация `/openapi/v1.json` должна быть невозможна в принципе вне профиля разработки – например, регистрировать маршрут только при явно заданном отдельном флаге конфигурации, а не полагаться на то, что переменная окружения `ASPNETCORE_ENVIRONMENT` никогда не попадёт в эксплуатацию по ошибке.

б) **Добавить заголовки Strict-Transport-Security и Content-Security-Policy.** Первый – со сроком действия не менее года и директивой `includeSubDomains`; второй – как минимум с политикой по умолчанию `default-src 'self'`, дополнительно снижающей ущерб от XSS даже в источниках, которые пока не рассматривались.

в) **Терминировать TLS перед приложением или в нём самом.** Развёртывание должно включать реверс-прокси (nginx, Caddy) или сертификат Kestrel так, чтобы `UseHttpsRedirection` действительно имел куда перенаправлять, а не молчаливо не срабатывал.

г) **Скрыть или заменить заголовок Server.** Отключение баннера сервера – стандартная и дешёвая мера снижения информативности разведки для атакующего.

д) **Ввести серверное экранирование/санитизацию сохраняемого содержимого.** Даже при доверии к текущему клиенту, вывод API не должен полагаться на то, что ни один клиент никогда не отобразит данные как HTML – следует либо экранировать управляющие символы при выдаче, либо явно документировать и проверять на уровне контракта API, что поле является неэкранированным сырым текстом.

е) **Распространить ограничение частоты запросов на /api/auth/register.** Тот же механизм блокировки, что уже защищает `/api/auth/login`, должен применяться к регистрации, иначе перечисление существующих имён пользователей ничем не ограничено.

ж) **Разделить ключи по областям ответственности.** Ключ подписи JWT и ключ шифрования конфиденциальных данных должны быть независимыми секретами с независимой ротацией, чтобы компрометация одного не означала автоматической компрометации другого.

з) **Добавить журналирование событий безопасности.** Неудачные попытки входа, срабатывания блокировки и факты регистрации должны фиксироваться в структурированном логе (с указанием времени и адреса источника, но без самого пароля) – без этого механизм блокировки работает, но остаётся невидимым для последующего расследования инцидента.

и) **Вынести состояние блокировки попыток входа в общее хранилище.** Текущая реализация хранит счётчик неудачных попыток в памяти одного процесса; при масштабировании на несколько экземпляров

за балансировщиком нагрузки это ограничение перестанет работать надёжно и должно быть заменено на общее хранилище (например, Redis).

Рекомендации по безопасному использованию приложения

а) Никогда не запускать приложение в реальной эксплуатации с `ASPNETCORE_ENVIRONMENT=Development` – этот режим, помимо открытой документации API, включает подробные страницы ошибок, раскрывающие внутреннее устройство приложения.

б) Никогда не переносить в эксплуатацию ключи из `appsettings.Development.json` – они предназначены только для локальной разработки, присутствуют в открытом виде в системе контроля версий и должны считаться скомпрометированными по определению; в эксплуатации ключи задаются переменными окружения или менеджером секретов и не совпадают с ключами разработки ни в одном окружении.

в) Размещать приложение только за узлом терминирования TLS с действующим сертификатом и не открывать порт API напрямую в недоверенную сеть.

г) Ограничить сетевую доступность порта базы данных/файла хранилища средствами ОС или контейнеризации – прямой доступ к файлу обходит всю описанную выше модель контроля доступа на уровне API.

д) Мониторировать код ответа 429 на `/api/auth/login` как индикатор возможной попытки подбора пароля до тех пор, пока не реализовано отдельное журналирование (см. меру 8 выше).

е) Использовать для учётных записей пароли, соответствующие как минимум установленному приложением порогу длины, но по возможности превышающие его – механизм не проверяет пароль на вхождение в списки скомпрометированных паролей и полагается на длину и уникальность, выбранные пользователем.

ж) Периодически (не реже, чем при каждом релизе) повторять протокол `audit.sh` как регрессионную проверку защищённости – часть найденных в этой работе замечаний (открытая документация API, отсутствующие заголовки) относится к классу конфигурационных ошибок, которые могут случайно вернуться при последующих изменениях инфраструктуры.

Выводы

В ходе выполнения лабораторной работы была разработана методика оценки защищённости сторонних программных систем, охватывающая все семь направлений анализа из теоретических основ задания, и применена на практике к реальному веб-приложению – варианту лабораторной работы №2.1 – с позиции внешнего наблюдателя, без изначального доверия к исходному коду. По результатам 26 автоматизированных проверок обнаружено шесть замечаний (открытая документация API, отсутствие двух заголовков транспортной безопасности, фактически неработающее перенаправление на HTTPS, отсутствие ограничения частоты запросов на регистрации, отсутствие серверного экранирования сохраняемого содержимого), для каждого из которых предложена конкретная мера усовершенствования, а также сформулирован отдельный набор рекомендаций по безопасной эксплуатации приложения, не требующих изменения кода.

Семантически работа показала принципиальное отличие оценки защищённости «снаружи» от анализа «изнутри», выполненного в лабораторной работе №2.1: разработчик системы знает, что определённый маршрут документации API должен быть недоступен в эксплуатации, потому что так задумано в коде; внешний наблюдатель этого не знает и обнаруживает лишь наблюдаемый факт – маршрут отвечает или не отвечает, – и именно это несоответствие между тем, что «задумано», и тем, что «реально доступно по сети», регулярно оказывается источником уязвимостей в эксплуатации реальных систем, а не ошибки в самом алгоритме шифрования или хеширования. Отдельно подтверждено, что часть свойств системы (SQL-инъекции, устойчивость JWT, шифрование данных) успешно выдержала проверку именно потому, что эти свойства были заложены разработчиком осознанно в лабораторной работе №2.1, тогда как обнаруженные замечания относятся преимущественно к конфигурации развёртывания и к неполной симметрии между схожими по назначению конечными точками (`login` и `register`), а не к отсутствию защитных механизмов как таковых.

С прагматической точки зрения работа показала, что содержательный аудит защищённости не требует непременно тяжёлых готовых сканеров – воспроизводимый набор небольших целевых скриптов, реализующих те же категории проверок (разведка ресурсов, инъекционное тестирование, проверка заголовков, проверка сессии), дал конкретные, проверяемые и привязанные к реальному HTTP-ответу замечания, которые можно использовать как регрессионный набор проверок при последующих изменениях приложения. Также показано, что не всякое найденное отклонение является уязвимостью в строгом смысле: находка по XSS была сознательно классифицирована как «не эксплуатируется текущим клиентом, но является скрытым риском», а не как подтверждённая уязвимость, поскольку

выданная API полезная нагрузка была дополнительно проверена против исходного кода конкретного клиента – корректная оценка защищённости требует различать эти две степени серьёзности, а не приравнивать любое отклонение от эталонного поведения к атаке.